

Documentação Técnica - Integração PlugQi API

Especificações Técnicas

Arquitetura do Sistema

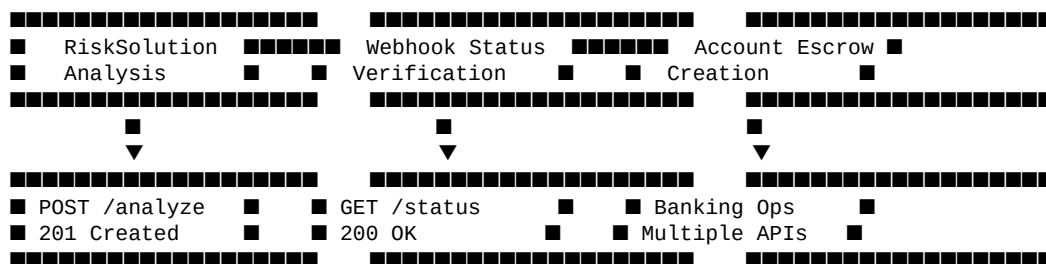
- **SDK**: Python 3.8+
- **Autenticação**: JWT ES512 com chaves ECDSA P-521
- **Protocolo**: HTTPS/REST API
- **Formato**: JSON
- **Encoding**: UTF-8

Dependências Core

```
# requirements.txt
requests>=2.28.0
cryptography>=3.4.8
PyJWT>=2.6.0
python-dotenv>=0.19.0
```

Arquitetura de Integração

Fluxo de Estados



Implementação de Autenticação

JWT ES512 Token Generation

```
import jwt
from cryptography.hazmat.primitives import serialization
from datetime import datetime, timedelta

class QiTechAuth:
    def __init__(self, private_key_path: str, client_key: str):
        self.client_key = client_key
        self.private_key = self._load_private_key(private_key_path)
```

```

def _load_private_key(self, path: str):
    with open(path, 'rb') as key_file:
        return serialization.load_pem_private_key(
            key_file.read(),
            password=None
        )

def generate_token(self, method: str, endpoint: str, body: str = "") -> str:
    payload = {
        'iss': self.client_key,
        'sub': self.client_key,
        'aud': 'qitech.com.br',
        'iat': datetime.utcnow(),
        'exp': datetime.utcnow() + timedelta(minutes=5),
        'method': method.upper(),
        'uri': endpoint,
        'body': body
    }

    return jwt.encode(
        payload,
        self.private_key,
        algorithm='ES512',
        headers={'typ': 'JWT', 'alg': 'ES512'}
    )

```

RiskSolution Integration

API Endpoints

Base URL: <https://risk-api.pluginz.com.br>
 Authentication: Bearer Token
 Content-Type: application/json

Natural Person Payload Schema

```

from typing import Dict, List, Optional
from dataclasses import dataclass
from datetime import datetime

@dataclass
class NaturalPersonRequest:
    id: str
    registration_id: str
    registration_date: str # ISO 8601
    client_category: str = "individual"
    name: str
    document_number: str # CPF format: XXX.XXX.XXX-XX
    birthdate: str # YYYY-MM-DD
    gender: str # male|female|other
    nationality: str = "BRA"
    mother_name: str
    father_name: Optional[str]
    monthly_income: int # Centavos
    declared_assets: int # Centavos
    occupation: str
    emails: List[Dict[str, str]]
    phones: List[Dict[str, str]]
    address: Dict[str, str]
    source: Dict[str, str]

# Validation Rules
VALIDATION_RULES = {
    'document_number': r'^\d{3}\.\d{3}\.\d{3}-\d{2}$',
    'birthdate': r'^\d{4}-\d{2}-\d{2}$',

```

```

        'postal_code': r'^\d{5}-\d{3}$',
        'phone_number': r'^\d{8,9}$',
        'area_code': r'^\d{2}$'
    }
}

```

Legal Person Payload Schema

```

@dataclass
class LegalPersonRequest:
    id: str
    registration_id: str
    registration_date: str
    client_category: str = "company"
    legal_name: str
    trading_name: str
    document_number: str # CNPJ format: XX.XXX.XXX/XXXX-XX
    foundation_date: str
    website: Optional[str]
    activity: str
    activity_code: str # CNAE
    merchant_category_code: str # MCC
    tier: str # micro|small|medium|large
    annual_revenues: int # Centavos
    emails: List[Dict[str, str]]
    phones: List[Dict[str, str]]
    address: Dict[str, str]
    source: Dict[str, str]
    partners: List[Dict[str, str]]

```

HTTP Implementation

```

import requests
import json
from typing import Union

class RiskSolutionClient:
    def __init__(self, api_key: str, base_url: str = "https://risk-api.pluginz.com.br"):
        self.api_key = api_key
        self.base_url = base_url
        self.session = requests.Session()
        self.session.headers.update({
            'Authorization': f'Bearer {api_key}',
            'Content-Type': 'application/json',
            'User-Agent': 'PlugQi-SDK/1.0'
        })

    def send_natural_person(self, data: Dict, analyze: bool = True) -> requests.Response:
        endpoint = f"{self.base_url}/v1/natural-person"
        params = {'analyze': str(analyze).lower()}

        response = self.session.post(
            endpoint,
            json=data,
            params=params,
            timeout=30
        )

        if response.status_code not in [200, 201]:
            raise Exception(f"API Error: {response.status_code} - {response.text}")

        return response

    def send_legal_person(self, data: Dict, analyze: bool = True) -> requests.Response:

```

```

endpoint = f"{self.base_url}/v1/legal-person"
params = {'analyze': str(analyze).lower()}

return self.session.post(endpoint, json=data, params=params, timeout=30)

```

Webhook Status Verification

Endpoint Specification

GET <https://qi.homolog.plugz.com.br/webhooks/qitech/status>
 Query Parameters:
 - id_registro: string (required) - Registration ID from RiskSolution
 Headers:
 - Authorization: Bearer <token>
 - Content-Type: application/json

Response Schema

```

from enum import Enum
from typing import Optional

class AnalysisStatus(Enum):
    PENDING = "pending"
    APPROVED = "approved"
    REJECTED = "rejected"
    UNDER_REVIEW = "under_review"

@dataclass
class StatusResponse:
    status: AnalysisStatus
    registration_id: str
    analysis_result: Optional[Dict]
    timestamp: str
    risk_score: Optional[int]
    recommendation: Optional[str]
    details: Optional[str]

```

Implementation with Retry Logic

```

import time
from typing import Optional

class WebhookStatusChecker:
    def __init__(self, base_url: str, auth_token: str):
        self.base_url = base_url
        self.auth_token = auth_token
        self.session = requests.Session()
        self.session.headers.update({
            'Authorization': f'Bearer {auth_token}',
            'Content-Type': 'application/json'
        })

    def check_status(self, registration_id: str, max_retries: int = 5) -> Dict:
        endpoint = f"{self.base_url}/webhooks/qitech/status"
        params = {'id_registro': registration_id}

        for attempt in range(max_retries):
            try:
                response = self.session.get(endpoint, params=params, timeout=10)

                if response.status_code == 200:
                    return response.json()
                elif response.status_code == 404:

```

```

        time.sleep(2 ** attempt) # Exponential backoff
        continue
    else:
        raise Exception(f"HTTP {response.status_code}: {response.text}")

except requests.RequestException as e:
    if attempt == max_retries - 1:
        raise
    time.sleep(2 ** attempt)

raise Exception(f"Max retries exceeded for registration_id: {registration_id}")

```

QiTech Account Creation

PlugQi SDK Integration

```

from plugqi import PlugQi
from typing import Dict, List

class AccountCreationService:
    def __init__(self):
        self.plugqi = PlugQi()
        self._validate_connection()

    def _validate_connection(self):
        if not self.plugqi.health_check():
            raise ConnectionError("QiTech API connection failed")

    def create_pf_account(self, analysis_result: Dict) -> Dict:
        """Create Natural Person account after approval"""

        person_data = self.plugqi.account_orchestrator.build_person_data(
            name=analysis_result['name'],
            document=analysis_result['document_number'].replace('.', '').replace(
                ('-', '')),
            email=analysis_result['emails'][0]['email'],
            birthdate=analysis_result['birthdate']
        )

        # Document paths - must be provided by client
        documents_paths = {
            "rg_front": analysis_result.get('rg_front_path'),
            "rg_back": analysis_result.get('rg_back_path'),
            "proof_residence": analysis_result.get('proof_residence_path')
        }

        return self.plugqi.account_orchestrator.create_account_pf_complete(
            person_data=person_data,
            documents_paths=documents_paths
        )

    def create_pj_account(self, analysis_result: Dict) -> Dict:
        """Create Legal Person account after approval"""

        company_data = self.plugqi.account_orchestrator.build_company_data(
            legal_name=analysis_result['legal_name'],
            trading_name=analysis_result['trading_name'],
            document=analysis_result['document_number'].replace('.', '').replace(
                ('/', '').replace('-', '')),
            email=analysis_result['emails'][0]['email']
        )

        return self.plugqi.account_orchestrator.create_account_pj_complete(
            company_data=company_data,
            documents_paths=analysis_result.get('documents_paths', {})
        )

```

)

Banking Operations Implementation

Boleto Creation

```
class BoletoService:
    def __init__(self, plugqi_instance):
        self.plugqi = plugqi_instance

    def create_boleto(self, account_key: str, boleto_data: Dict) -> Dict:
        """Create boleto with validation"""

        # Validate required fields
        required_fields = ['amount', 'expiration', 'payer_data']
        for field in required_fields:
            if field not in boleto_data:
                raise ValueError(f"Missing required field: {field}")

        # Generate unique control key
        boleto_data['request_control_key'] = str(uuid.uuid4())

        return self.plugqi.boleto.create_boleto(
            account_key=account_key,
            requester_profile_key=self._get_profile_key(account_key),
            boleto_data=boleto_data
        )
```

PIX Operations

```
class PIXService:
    def __init__(self, plugqi_instance):
        self.plugqi = plugqi_instance

    def send_pix(self, account_key: str, pix_key: str, amount: float, description: str) -> Dict:
        """Send PIX with validation"""

        # Validate PIX key format
        if not self._validate_pix_key(pix_key):
            raise ValueError(f"Invalid PIX key format: {pix_key}")

        return self.plugqi.pix.enviar_pix_chave(
            account_key=account_key,
            pix_key=pix_key,
            valor=amount,
            descricao=description
        )

    def _validate_pix_key(self, pix_key: str) -> bool:
        """Validate PIX key format"""
        import re

        patterns = {
            'email': r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
            'cpf': r'^\d{11}$',
            'cnpj': r'^\d{14}$',
            'phone': r'^\+55\d{10,11}$',
            'random': r'^[a-f0-9]{8}-[a-f0-9]{4}-[a-f0-9]{4}-[a-f0-9]{4}-[a-f0-9]{12}$'
        }

        return any(re.match(pattern, pix_key) for pattern in patterns.values())
```

Error Handling Monitoring

Exception Hierarchy

```
class PlugQiException(Exception):
    """Base exception for PlugQi operations"""
    pass

class AuthenticationError(PlugQiException):
    """JWT authentication failed"""
    pass

class ValidationError(PlugQiException):
    """Data validation failed"""
    pass

class APIError(PlugQiException):
    """QiTech API returned error"""
    def __init__(self, status_code: int, message: str, response_data: Dict = None):
        self.status_code = status_code
        self.message = message
        self.response_data = response_data
        super().__init__(f"API Error {status_code}: {message}")
```

Logging Configuration

```
import logging
import json
from datetime import datetime

class PlugQiLogger:
    def __init__(self, name: str = "plugqi"):
        self.logger = logging.getLogger(name)
        self.logger.setLevel(logging.INFO)

        # JSON formatter for structured logging
        formatter = logging.Formatter(
            '{"timestamp": "%(asctime)s", "level": "%(levelname)s", '
            '"module": "%(name)s", "message": "%(message)s"}'
        )

        handler = logging.StreamHandler()
        handler.setFormatter(formatter)
        self.logger.addHandler(handler)

    def log_api_call(self, method: str, endpoint: str, status_code: int,
                    duration: float, request_id: str = None):
        self.logger.info(json.dumps({
            "event": "api_call",
            "method": method,
            "endpoint": endpoint,
            "status_code": status_code,
            "duration_ms": round(duration * 1000, 2),
            "request_id": request_id
        })))
```

Testing Strategy

Unit Tests

```
import pytest
from unittest.mock import Mock, patch
```

```

class TestRiskSolutionIntegration:
    @pytest.fixture
    def risk_client(self):
        return RiskSolutionClient(api_key="test-key")

    @patch('requests.Session.post')
    def test_send_natural_person_success(self, mock_post, risk_client):
        # Mock successful response
        mock_response = Mock()
        mock_response.status_code = 201
        mock_response.json.return_value = {"id": "test-id", "status": "created"}
        mock_post.return_value = mock_response

        # Test data
        person_data = {
            "id": "test-id",
            "name": "Test Person",
            "document_number": "123.456.789-01"
        }

        # Execute
        response = risk_client.send_natural_person(person_data)

        # Assertions
        assert response.status_code == 201
        mock_post.assert_called_once()

```

Integration Tests

```

class TestFullIntegrationFlow:
    def test_complete_flow(self):
        """Test complete integration flow"""

        # 1. Send to RiskSolution
        risk_client = RiskSolutionClient(api_key=os.getenv('RISK_API_KEY'))
        person_data = self._build_test_person_data()

        risk_response = risk_client.send_natural_person(person_data, analyze=False)
        assert risk_response.status_code == 201

        registration_id = risk_response.json()['registration_id']

        # 2. Check status (with polling)
        status_checker = WebhookStatusChecker(
            base_url="https://qi.homolog.plugz.com.br",
            auth_token=os.getenv('WEBHOOK_TOKEN')
        )

        status = status_checker.check_status(registration_id)
        assert status['status'] == 'approved'

        # 3. Create account
        account_service = AccountCreationService()
        account_result = account_service.create_pf_account(status)

        assert 'account_key' in account_result
        assert account_result['status'] == 'success'

```

Security Considerations

Key Management

```

import os
from cryptography.fernet import Fernet

```



```

class SecureKeyManager:
    def __init__(self):
        self.encrypted_key = os.getenv('ENCRYPTION_KEY', Fernet.generate_key())
        self.cipher = Fernet(self.encrypted_key)

    def encrypt_sensitive_data(self, data: str) -> str:
        return self.cipher.encrypt(data.encode()).decode()

    def decrypt_sensitive_data(self, encrypted_data: str) -> str:
        return self.cipher.decrypt(encrypted_data.encode()).decode()

```

Rate Limiting

```

import time
from collections import defaultdict

class RateLimiter:
    def __init__(self, max_requests: int = 100, window_seconds: int = 60):
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self.requests = defaultdict(list)

    def is_allowed(self, client_id: str) -> bool:
        now = time.time()
        client_requests = self.requests[client_id]

        # Remove old requests
        client_requests[:] = [req_time for req_time in client_requests
                               if now - req_time < self.window_seconds]

        if len(client_requests) >= self.max_requests:
            return False

        client_requests.append(now)
        return True

```

Performance Optimization

Connection Pooling

```

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

class OptimizedHTTPClient:
    def __init__(self):
        self.session = requests.Session()

        # Retry strategy
        retry_strategy = Retry(
            total=3,
            backoff_factor=1,
            status_forcelist=[429, 500, 502, 503, 504]
        )

        # HTTP adapter with connection pooling
        adapter = HTTPAdapter(
            pool_connections=10,
            pool_maxsize=20,
            max_retries=retry_strategy
        )

        self.session.mount("http://", adapter)
        self.session.mount("https://", adapter)

```

Deployment Configuration

Environment Variables

```
# .env.production
QITECH_CLIENT_KEY=prod-client-key
QITECH_PRIVATE_KEY_PATH=/secure/keys/ec_p521_private.pem
QITECH_BASE_URL=https://api-auth.qitech.app
RISK_SOLUTION_API_KEY=prod-risk-key
WEBHOOK_BASE_URL=https://qi.plugz.com.br
LOG_LEVEL=INFO
REDIS_URL=redis://localhost:6379/0
DATABASE_URL=postgresql://user:pass@localhost/plugqi
```

Docker Configuration

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "app:app"]
```

API Reference Summary

Endpoint	Method	Purpose	Auth Required
/v1/natural-person`	POST	Submit PF for analysis	Bearer Token
/v1/legal-person`	POST	Submit PJ for analysis	Bearer Token
/webhooks/qitech/status`	GET	Check analysis status	Bearer Token
/qitech/account/create`	POST	Create escrow account	JWT ES512
/qitech/boleto/create`	POST	Generate boleto	JWT ES512
/qitech/pix/send`	POST	Send PIX payment	JWT ES512

Troubleshooting Guide

Common Issues

- JWT Token Expired**: Regenerate token with current timestamp
- Invalid Key Format**: Ensure ECDSA P-521 curve
- Webhook Timeout**: Implement exponential backoff
- Rate Limiting**: Implement request queuing

5. ****SSL Certificate****: Verify certificate chain

Debug Mode

```
import logging
logging.basicConfig(level=logging.DEBUG)

# Enable HTTP request logging
import http.client as http_client
http_client.HTTPConnection.debuglevel = 1
```